



MoveStream

Prototipo de aplicación multimodal

TÉCNICAS AVANZADAS DE INTERACCIÓN
MULTIMODAL

Nataly Rocha

Universidad de Valladolid

Escuela de Ingeniería Informática

2025 – 2026

1 Motivación del Proyecto

Los streamers actuales controlan sus efectos visuales mediante teclado, StreamDeck y botones físicos durante sus transmisiones en directo. Esta dependencia en dispositivos físicos genera problemas que afectan directamente a la calidad del contenido:

- **Ruptura del contacto visual:** el streamer debe apartar la vista de la cámara para localizar y pulsar el botón correcto en momentos clave.
- **Pérdida de autenticidad:** la reacción genuina queda interrumpida mientras se busca el control, restando naturalidad al contenido.
- **Complejidad técnica:** las producciones más elaboradas requieren un técnico adicional para gestionar efectos en tiempo real.
- **Barrera de acceso:** equipo especializado como el StreamDeck supone una inversión económica que no está al alcance de todos los creadores de contenido.

La disponibilidad de APIs de visión computacional de alto rendimiento en el navegador (Google MediaPipe [?]) y de reconocimiento de voz nativo (Web Speech API [?]) abre la posibilidad de crear interfaces gestuales sin hardware adicional. Es este problema el que motiva el desarrollo de MoveStream.

2 Objetivo del Proyecto

El objetivo principal es desarrollar **MoveStream**: un prototipo de asistente de efectos visuales para streaming que permita activar overlays y efectos en tiempo real mediante gestos de mano, expresiones faciales y comandos de voz, sin tocar ningún dispositivo durante la transmisión.

Los objetivos específicos son:

- Implementar reconocimiento de gestos de mano en tiempo real con MediaPipe HandLandmarker.
- Detectar expresiones faciales mediante MediaPipe FaceLandmarker y análisis de blendshapes.
- Integrar reconocimiento de voz con la Web Speech API, condicionado a un gesto activador.
- Renderizar efectos visuales y overlays sobre el vídeo de la cámara usando Three.js [?].
- Posibilitar la integración con OBS Studio [?] sin hardware adicional.

La aplicación se ejecuta íntegramente en el navegador web, sin backend, usando únicamente una webcam estándar y el micrófono del sistema.

3 Modalidades de Entrada/Salida

3.1 Entradas

3.1.1 Gestos de Mano — MediaPipe HandLandmarker

MediaPipe HandLandmarker detecta hasta dos manos simultáneamente, devolviendo 21 landmarks 3D normalizados por mano y la etiqueta de lateralidad (*Left/Right*). Sobre estos datos se implementaron tres detectores personalizados:

Gesto	Mecanismo de detección	Efecto activado
Puño izquierdo + todos los dedos de la derecha extendidos	<code>verticalHandDetector</code> : requiere exactamente 2 manos con handedness correcto	Portal Dr. Strange (swirl shader)
Triángulo índice-pulgar (proximidad)	<code>triangleDetector</code> : distancia entre landmarks	Tercer Ojo (aura verde + ojo 3D + parallax)
Símbolo de paz (V) sostenido 400 ms	<code>lGestureDetector</code> : hold timer sobre mano izquierda	Activa escucha de voz

Table 1: Gestos de mano implementados

3.1.2 Voz — Web Speech API

El reconocimiento de voz se activa exclusivamente cuando el gesto de paz está activo, evitando activaciones accidentales. Se reconocen tres frases clave configurables:

- “**Chisme Potente**” → efecto de glitch visual + audio de corneta.
- “**Caliente**” → overlay dramático con pulso naranja/rojo + audio de explosión.
- “**Super Elegante**” → meme Ariel en burbuja dorada animada.

3.1.3 Expresiones Faciales — MediaPipe FaceLandmarker

FaceLandmarker devuelve 478 landmarks faciales y 52 blendshapes con valores $[0, 1]$ que describen la geometría facial. El `expressionDetector` analiza combinaciones de blendshapes para identificar cinco expresiones:

Expresión	Blendshapes clave	Efecto
Escéptico	<code>eyeSquintLeft/Right</code> alto + <code>browDown</code> alto + boca cerrada	Spider Sense (wireframe vibrante)
Risa malévola	<code>jawOpen</code> > 0.2 + <code>mouthSmile</code> > 0.4	Meme risaMalevola
Sorprendido	<code>eyeWideLeft/Right</code> muy alto	Meme sorprendido
Preocupado	<code>browInnerUp</code> alto + boca cerrada	Meme preocupado
Smug	Comisuras bajas + boca cerrada	Meme smug

Table 2: Expresiones faciales detectadas

3.2 Salidas

- **Efectos 3D con Three.js**: portal con shader GLSL, aura verde con ojo 3D, wireframe Spider-Sense, ojos flotantes con parallax facial.
- **Overlays 2D con React/CSS**: memes en burbuja circular dorada animada, overlay *caliente* con animación de escala y brillo.
- **Audio con Web Audio API**: efectos pre-decodificados (corneta, explosión, Spider-Sense) con fade-out suave.

- **Post-proceso con EffectComposer:** GlitchPass aplicado a toda la escena para el efecto “Chisme Potente”.

4 Boceto de la Aplicación

4.1 Diagrama de Arquitectura

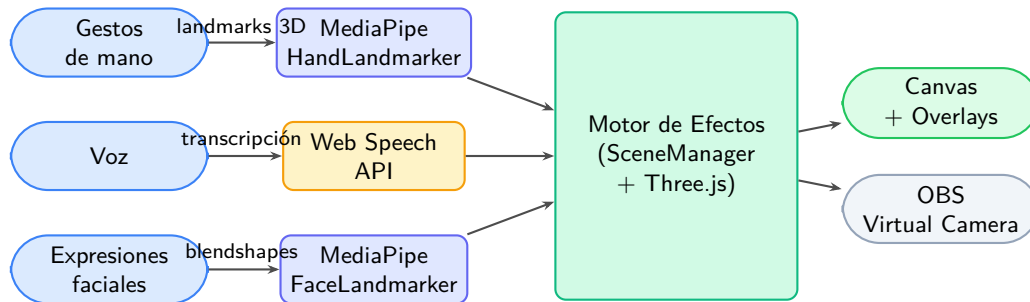


Figure 1: Arquitectura de procesamiento multimodal de MoveStream

4.2 Pantallas Principales

La interfaz consta de dos pantallas:

1. **Landing page:** nebulosa de 5000 partículas Three.js con colores del tema (violetas y azules), título animado “MOVESTREAM” y botón “Empezar Stream”.
2. **Pantalla principal:** canvas Three.js con el vídeo de la webcam como textura de fondo, HUD de chips de estado en la parte inferior (modalidades activas), panel lateral con historial de interacciones y los overlays React superpuestos.

5 Descripción de la Propuesta Desarrollada

5.1 Stack Tecnológico

MoveStream es una aplicación web sin backend construida con React 18 [?] y Vite. El stack completo es:

- **React 18 + Vite:** scaffolding, UI declarativa y gestión de hooks.
- **Three.js 0.184:** renderizado 3D, shaders GLSL personalizados, EffectComposer.
- **@mediapipe/tasks-vision:** HandLandmarker y FaceLandmarker ejecutados en WASM + WebGL.
- **Web Speech API:** reconocimiento de voz nativo del navegador.

5.2 Arquitectura del Código

La arquitectura utiliza el patrón de custom hooks React, cada uno encapsulando una modalidad de entrada:

- **useHandTracking:** ejecuta el loop de detección de manos a ≈ 30 fps y llama a los tres detectores de gestos.
- **useFaceTracking:** gestiona FaceLandmarker, llama a `expressionDetector` y activa memes o Spider-Sense.

- **useSpeechRecognition**: encapsula Web Speech API; solo escucha cuando el gesto de paz está activo.
- **useThreeScene**: inicializa y conecta el **SceneManager** con el ciclo de vida React.

El estado global se comparte mediante **AppContext**, que contiene los efectos activos y referencias de alta frecuencia a los landmarks actuales. Todos los efectos Three.js siguen una interfaz uniforme: `constructor()`, `init(scene)`, `update(deltaTime, landmarks)`, `setActive(bool)`, `dispose()`.

5.3 Efectos Implementados

5.3.1 Portal Dr. Strange

El gesto bimanual activa **PortalEffect**: un disco Three.js con un shader GLSL de torbellino (*swirl*). El shader anima la distorsión radial de una textura de póster de película en función del tiempo mediante uniforms, creando el efecto característico del portal del Doctor Extraño. Un anillo exterior de partículas rodea el disco.

5.3.2 Tercer Ojo (Snap Aura)

La aproximación de índice y pulgar activa dos efectos combinados: **SnapAuraEffect** muestra una esfera brillante verde con un ojo 3D animado posicionado sobre el rostro del usuario (posición calculada convirtiendo landmarks de MediaPipe a coordenadas del mundo Three.js con `normalizedToWorld()`), y **ParallaxEffect** genera ojos flotantes que siguen el movimiento de la cabeza con efecto de parallax facial.

5.3.3 Spider Sense

La expresión escéptica (debounce de 800ms) activa **SpiderSenseEffect**: líneas de energía generadas desde landmarks distribuidos de la frente del usuario hacia el exterior del canvas, animadas con oscilación sinusoidal. Incluye audio de activación y fade-out suave al desactivarse.

5.3.4 Efectos de Voz

Los tres comandos producen efectos diferenciados:

- **Chisme Potente**: aplica **GlitchPass** del **EffectComposer** a toda la escena durante 3 segundos, acompañado de audio de corneta pre-decodificado con Web Audio API.
- **Caliente**: overlay CSS con imagen `hot.png`, animación *bounce-in* de escala y pulso de brillo naranja/rojo, más audio de explosión.
- **Super Elegante**: **MemeOverlay** con `ariel.png` en burbuja circular dorada con animación CSS en la esquina inferior derecha.

5.3.5 Memes Homelander

FaceLandmarker monitoriza cuatro expresiones de forma continua (con debounce individual). Al detectar cada expresión, **MemeOverlay** muestra la imagen correspondiente en la burbuja dorada. Solo puede estar activo un meme simultáneamente; el más reciente sustituye al anterior.

5.4 Integración con OBS

La aplicación está diseñada para su uso con OBS Studio a través de la funcionalidad de Virtual Camera. El streamer abre la aplicación en Chrome (recomendado por su soporte completo de

Web Speech API), activa la Virtual Camera del navegador en OBS, y tiene el canvas completo con todos los overlays disponible como fuente de vídeo capturada.

6 Dificultades Encontradas

6.1 Detección de Gestos de Mano

Falsos positivos en el portal. La primera versión usaba un único detector de mano vertical que se activaba con movimientos cotidianos. Se resolvió cambiando a un gesto bimanual que requiere simultáneamente un puño cerrado en la izquierda y todos los dedos extendidos en la derecha, reduciendo drásticamente los falsos positivos al exigir dos condiciones independientes.

Handedness invertido. Con cámara frontal en modo espejo, los labels *Left/Right* de MediaPipe están especulares respecto al usuario. Fue necesario intercambiar los labels manualmente en el detector para que el gesto del portal funcionara con la mano correcta.

Bug lógico en verticalHandDetector. La llamada a `detect()` estaba situada en el bloque `else` (sin manos presentes), por lo que nunca se ejecutaba cuando había manos detectadas. El portal nunca se activaba. Detectado mediante depuración con `console.log`.

Gesto L poco robusto. El gesto L original para activar la voz tenía alta tasa de falsos positivos. Se sustituyó por el símbolo de paz (dos dedos en V), con mejor cobertura en el modelo de MediaPipe.

Detector de triángulo desde cero. Requirió múltiples iteraciones con logs de distancias entre landmarks para calibrar los umbrales de activación correctos.

6.2 Detección de Expresiones Faciales

Umbral de risa demasiado estricto. El umbral inicial `jawOpen > 0.5` exigía una apertura bucal exagerada. Reducirlo a `jawOpen > 0.2` permitió capturar sonrisas naturales con dientes.

Spider-Sense con falsos positivos continuos. La expresión escéptica se detectaba en cualquier parpadeo. Se implementó un debounce de 800ms que exige sostener la expresión antes de lanzar el efecto. Este patrón resultó ser necesario para la mayoría de expresiones del sistema.

Ojo del Tercer Ojo en posición incorrecta. `SnapAuraEffect` leía `faceDetections[0].keypoints` (formato de la API antigua Face Detection) cuando la API de `FaceLandmarker` devuelve un array de 478 landmarks con estructura diferente. El ojo se posicionaba en (0,0,0) en lugar de sobre el rostro. La depuración requirió `console.log` del formato de los datos y ajuste empírico de los índices de landmarks hasta lograr la posición correcta.

Rayos del Spider-Sense superpuestos. Los landmarks de la cabeza están concentrados en la frente, generando rayos en posiciones muy similares que se solapan formando una X en lugar de distribuirse uniformemente. Se resolvió usando un subconjunto distribuido de landmarks con separación angular mínima entre rayos.

Grosor de línea en WebGL. La API de Three.js (`LineBasicMaterial`) no admite grosor de línea real en WebGL: siempre renderiza a 1px independientemente del valor configurado. Los rayos del Spider-Sense no tienen el grosor visual deseado; es una limitación de la plataforma no resoluble sin usar geometría de cilindros o librerías como `THREE.MeshLine`.

6.3 Audio y Post-proceso

Póster del portal invisible. `renderOrder = -1` situaba el disco del portal detrás del plano de vídeo de la webcam. Cambiado a `renderOrder = 1` para renderizarlo sobre la escena.

Portal no visible en el HUD. El estado `verticalHandActive` no estaba registrado en `ModalitiesPanel`, por lo que el HUD nunca mostraba su estado activo.

Audio ausente en Spider-Sense. El efecto se activaba y desactivaba abruptamente sin ningún audio. Se añadieron sonido de activación y transición de fade-out mediante Web Audio API.

6.4 Reconocimiento de Voz

Latencia inherente de la Web Speech API. La API introduce un retraso de ≈ 0.5 –1 s entre la pronunciación y la acción. Es una limitación conocida del navegador, reducible pero no eliminable sin usar reconocimiento local (Whisper.js u otras alternativas), lo que requeriría un backend o descargar modelos de gran tamaño al cliente.

6.5 Rol del .env en la Depuración

El archivo `.env` resultó fundamental para gestionar umbrales configurables (valores de `blend-shapes`, distancias de gestos, duraciones de efectos, flags de debug) sin necesidad de recompilar. Permitió iterar rápidamente sobre los valores óptimos de detección durante el desarrollo, y combinado con el patrón de `debounce`, fue la herramienta central para distinguir intenciones reales de movimientos accidentales.

7 Conclusiones

MoveStream demuestra que es posible construir una interfaz multimodal funcional para streaming completamente en el navegador, sin hardware adicional y con tecnologías de código abierto. Los tres canales de entrada operan de forma combinada y complementaria: el gesto de paz actúa como modificador que habilita el canal de voz, ilustrando el concepto de interacción multimodal cooperativa donde una modalidad puede condicionar y enriquecer a las demás.

Los principales aprendizajes del proyecto son:

- **Los umbrales son críticos:** la detección de gestos y expresiones requiere calibración empírica. Mantener umbrales en variables de entorno configurables es imprescindible para iterar con agilidad.
- **El debounce es imprescindible:** sin él, los efectos se activan continuamente ante cualquier movimiento involuntario, haciendo la experiencia inutilizable en la práctica.
- **Las limitaciones de plataforma importan:** WebGL no admite líneas con grosor real; la Web Speech API tiene latencia inherente. Conocer estas restricciones desde el inicio evita tiempo de depuración innecesario.
- **La multimodalidad aporta valor real:** la combinación de gestos, voz y expresiones faciales genera un vocabulario de interacción mucho más rico y natural que cualquier modalidad por separado, especialmente en el contexto del streaming donde la espontaneidad y la conexión con la audiencia son el valor central.

Como trabajo futuro sería valioso reducir la latencia de voz con reconocimiento local (Whisper.js), añadir calibración por usuario de los umbrales gestuales, y explorar la integración directa

con la API WebSocket de OBS para control bidireccional entre la aplicación y el software de transmisión.

A Código Fuente del Proyecto

Estructura principal del código fuente:

```
src/
  App.jsx                                - Raiz: toggle landing/app con useState
  context/AppContext.jsx                 - Estado global compartido
  hooks/
    useHandTracking.js                   - Loop deteccion manos (~30 fps)
    useFaceTracking.js                   - FaceLandmarker + expresiones
    useThreeScene.js                     - Inicializa SceneManager
    useSpeechRecognition.js               - Web Speech API condicional
  gestures/
    verticalHandDetector.js               - Portal (gesto bimanual)
    triangleDetector.js                   - Tercer Ojo (proximidad landmarks)
    lGestureDetector.js                   - Paz -> activa voz (hold 400 ms)
    expressionDetector.js                 - Blendshapes faciales (5 expresiones)
  three/
    SceneManager.js                       - Loop render + registro de efectos
    effects/
      PortalEffect.js                     - Swirl shader GLSL
      SnapAuraEffect.js                   - Aura verde + ojo 3D
      SpiderSenseEffect.js                 - Wireframe Spider-Sense + audio
      ParallaxEffect.js                   - Ojos flotantes con parallax facial
      ChismePotente.js                    - GlitchPass + audio pre-decoded
  components/
    LandingPage.jsx                       - Nebulosa de particulas Three.js
    MemeOverlay.jsx                       - Burbuja dorada (memes Homelander)
    CalienteOverlay.jsx                   - Overlay caliente CSS + audio
  utils/
    coordUtils.js                         - normalizedToWorld(), keypointCenter()
```